Machine Learning Project 2 copilot chat

User: Take a look at this. And just keep it in context for now, relay to me what you understand before we proceed.

CS 4375: Introduction to Machine Learning
Project 2: Learning Tree-Based Classifiers and Their Ensembles
In this project, you will evaluate tree-based classifiers and their ensemble methods as discussed in class.
You will use scikit-learn and perform the following experiments.
1 Datasets: Learning Boolean Functions
• Download the 15 datasets available on eLearning. Each dataset is divided into three subsets: the
training set, the validation set, and the test set. The datasets are in CSV format, where each row
represents an instance with attribute values separated by commas. The last attribute corresponds to
the class variable.
• Assume that all attributes take values from the domain {0, 1}.
• The datasets are synthetically generated by randomly sampling solutions and non-solutions from a
Boolean formula in conjunctive normal form (CNF). Solutions are labeled as class “1,” while nonsolutions are labeled as class “0.”
Example CNF Formula:
(X1 ∨ ¬X2) ∧ (¬X1 ∨ X3) ∧ (X2 ∨ ¬X3)
Here ∧ denotes logical AND, ∨ denotes logical OR, and ¬ denotes logical NOT (negation).
This formula has three clauses:
– Clause 1: (X1 ∨ ¬X2)
– Clause 2: (¬X1 ∨ X3)
– Clause 3: (X2 ∨ ¬X3)
A datapoint is an assignment of values to (X1, X2, X3), with the label given by whether all clauses
are satisfied:
– (X1 = 0, X2 = 0, X3 = 0): all clauses True ⇒ label = 1.
– (X1 = 1, X2 = 0, X3 = 1): clause 3 False ⇒ label = 0.
Thus, each datapoint corresponds to a Boolean assignment, and the class label is 1 if the CNF evaluates
to true, and 0 otherwise.
• Five CNF formulas were generated with 500 variables and varying numbers of clauses: 300, 500,

1000, 1500, and 1800 clauses (each clause containing exactly 3 literals). From each formula, 100, 1000, and 5000 positive (class=1) and negative (class=0) examples were sampled. Therefore there are a total of 15 different classification problems.

• Filenames follow a structured naming convention:

1

– train c[i] d[j].csv contains training data with j examples generated from a formula with i clauses.

– test c[i] d[j].csv and valid c[i] d[j].csv contain the corresponding test and validation sets.

– Example: train c500 d100.csv contains 100 examples from the formula with 500 clauses.

• Important: Do not mix datasets. For instance, do not train on train c500 d100.csv and test on test c500 d5000.csv.

2 Experiments

You will learn the following four tree based classifiers on each of the 15 datasets: 1) decision tree classifiers, 2) bagged decision tree classifiers, 3) random forest classifiers, and 4) boosted decision tree classifiers. You have to report the performance of each model on each dataset and then finally compare their overall performances.

2.1 Decision Tree Classifiers (20 points)

Train a sklearn.tree.DecisionTreeClassifier on each of the 15 datasets. Follow the procedure below carefully, as the same workflow will be adopted for all subsequent classifiers.

(a) Hyperparameter Tuning: Use an exhaustive grid search to tune hyperparameters using the provided validation set (do not perform k-fold cross-validation).

Explore combinations of the following hyperparameters:

• Splitting criterion (e.g., gini, entropy)

• Maximum tree depth (max depth)

• Minimum number of samples required to split an internal node (min samples split)

• Minimum number of samples required at a leaf node (min samples leaf)

Note that max depth=None corresponds to fully grown trees (i.e., no depth constraint) and represents the highest model complexity setting.

You must design a reasonable search space for each hyperparameter.

Reporting (concise summary only):
Due to the larger hyperparameter space, you are not required to report the full grid of results. Instead, for each dataset, summarize:
• The hyperparameter ranges explored and your justification
• The total number of candidate models evaluated
• The selected best hyperparameter configuration
• The corresponding validation accuracy
Briefly discuss how model complexity changes as these hyperparameters vary.
2
(b) Best Model Selection: Select the hyperparameter combination that achieves the highest validation accuracy and report the chosen configuration.
(Different students may obtain different best configurations.)
(c) Retraining and Evaluation: Combine the training and validation sets. Retrain the classifier using the selected hyperparameter configuration on this combined dataset. Evaluate the final model on the test set and report:
• Classification accuracy
• F1 score (weighted average)
Include these test results in Table 2, together with the results from the other classifiers.
Reflection: How do the selected hyperparameters affect model complexity and generalization performance? For example, how does increasing max depth or decreasing min samples split and min samples leaf influence bias and variance? Do fully grown trees (max depth=None) consistently perform best on the validation set, or do they tend to overfit? What patterns do you observe across datasets with different training sizes and clause complexities?
2.2 Bagged Decision Tree Classifiers (20 points)
Repeat the procedure from the Decision Tree section using sklearn.ensemble.BaggingClassifier with a DecisionTreeClassifier as the base estimator.
Hyperparameter Tuning Systematically vary the following hyperparameters:
• The number of estimators (n estimators) (e.g., 10, 20, 50, 100)
• The maximum depth (max depth) of the base decision trees
Design a small but meaningful grid that explores different ensemble sizes (varying n estimators)

and different levels of base learner complexity (varying max depth).

For each dataset:

• Report training and validation accuracy for all combinations of n estimators and max depth in a table similar to Table 1.

• Select the best model based on validation accuracy.

Retraining and Evaluation Using the selected hyperparameter configuration, combine the training and validation sets and retrain the final model. Evaluate this model on the test set and report the classification accuracy and F1 score in a table similar to Table 2.

Reflection: As the number of estimators increases, does validation performance consistently improve? Do deeper trees (e.g., fully grown trees) always perform better in bagging? What does this suggest about the interaction between ensemble size and base learner complexity?

3

Table 1: Validation Accuracy of Bagged Decision Trees for Different Values of n estimators and max depth

Dataset	10			20			50			100		
	max depth			max depth			max depth			max depth		
	5	10	None	5	10	None	5	10	None	5	10	None
c300 d100	–	–	–	–	–	–	–	–	–	–	–	–
c300 d1000	–	–	–	–	–	–	–	–	–	–	–	–
c300 d5000	–	–	–	–	–	–	–	–	–	–	–	–
c500 d100	–	–	–	–	–	–	–	–	–	–	–	–
c500 d1000	–	–	–	–	–	–	–	–	–	–	–	–
c500 d5000	–	–	–	–	–	–	–	–	–	–	–	–
c1000 d100	–	–	–	–	–	–	–	–	–	–	–	–
c1000 d1000	–	–	–	–	–	–	–	–	–	–	–	–
c1000 d5000	–	–	–	–	–	–	–	–	–	–	–	–
c1500 d100	–	–	–	–	–	–	–	–	–	–	–	–
c1500 d1000	–	–	–	–	–	–	–	–	–	–	–	–
c1500 d5000	–	–	–	–	–	–	–	–	–	–	–	–

c1800 d100 – – – – – – – – – – – –
c1800 d1000 – – – – – – – – – – – –
c1800 d5000 – – – – – – – – – – – –

2.3 Random Forest Classifier (20 points)

Repeat the full procedure (hyperparameter tuning, model selection, and final evaluation) using sklearn.ensemble.RandomForestClassifier.

Hyperparameter Tuning Systematically vary the following hyperparameters:

• The number of estimators (n estimators) (e.g., 10, 20, 50, 100)

• The number of features considered at each split (max features), such as sqrt, log2, or a fraction (e.g., 0.5)

Design a reasonable grid that explores both ensemble size and the strength of feature subsampling. You may keep max depth=None, meaning all trees are fully grown.

For each dataset:

• Report training and validation accuracy and F1 scores across all combinations of n estimators and max features in a table similar to Table 1.

• Select the best model based on validation accuracy.

Retraining and Evaluation Using the selected hyperparameter configuration, combine the training and validation sets, retrain the final model, and report test set accuracy and F1 score in a table similar to Table 2.

Understanding max features: This parameter controls how many features each tree may consider when searching for the best split. Restricting features reduces correlation between trees. Since variance reduction depends on averaging relatively uncorrelated models, stronger feature subsampling can sometimes improve generalization.

Reflection: As the number of estimators increases, does validation performance consistently improve? How does changing max features affect performance? Is there a trade-off between feature restriction and model strength?

2.4 Boosted Classifiers (20 points)

Repeat the full procedure using sklearn.ensemble.AdaBoostClassifier.

Hyperparameter Tuning Systematically vary:

• The number of boosting rounds (n estimators) (e.g., 10, 20, 50, 100)

• The maximum depth of the base decision tree learners (e.g., 1, 3, 5)

Design a small but meaningful grid that explores both the number of boosting stages and the strength of the base learners.

For each dataset:

• Report training and validation accuracy across all combinations of n estimators and base learner depth in a table similar to Table 1.

• Select the best model based on validation accuracy.

Retraining and Evaluation Using the selected hyperparameter configuration, combine the training and validation sets, retrain the final model, and report test set accuracy and F1 score in a table similar to Table 2.

Conceptual Note: In AdaBoost, models are added sequentially, with each new learner placing more emphasis on previously misclassified examples. Increasing n estimators increases overall model capacity, while increasing tree depth makes each weak learner stronger.

Reflection: Does increasing the number of estimators always improve validation performance? Do deeper base learners necessarily perform better in boosting, or can overly strong learners lead to overfitting?

2.5 Comparative Analysis (20 points)

Using the best hyperparameter configuration selected via validation for each classifier, report the test set accuracy and F1 score for all datasets.

Present results in two separate tables (one for accuracy, one for F1), following the format below.

Answer the following questions.

• Which classifier achieves the best overall generalization performance? Is the ranking consistent across datasets? Explain.

• How does increasing the training data size impact performance for each classifier? Which models benefit most from more data?

• How does increasing the number of clauses affect performance? What does this suggest about the complexity of the underlying Boolean formula?

Table 2: Classification Accuracy on Test Set

Dataset	DecisionTree	Bagging	RandomForest	AdaBoost
c300 d100	–	–	–	–
c300 d1000	–	–	–	–

c300 d5000 – – – –
c500 d100 – – – –
c500 d1000 – – – –
c500 d5000 – – – –
c1000 d100 – – – –
c1000 d1000 – – – –
c1000 d5000 – – – –
c1500 d100 – – – –
c1500 d1000 – – – –
c1500 d5000 – – – –
c1800 d100 – – – –
c1800 d1000 – – – –
c1800 d5000 – – – –
Submission Instructions
Submit a single ZIP file containing:
• Your code, which demonstrates how the experiments were set up, and a README file with instructions for running the code.
• A report describing your results, including answers to all questions.
• Submit your AI chat transcript along with the project report (in the same format as Project 1).
Important: Your code must run without errors. If we cannot replicate your results, no credit will be given.

GitHub Copilot: Optimizing tool selection...# Project Summary - Understanding

This is a **tree-based classifiers project** where you need to evaluate and compare four classifier types on 15 synthetic Boolean learning datasets. Here's what I understand:

Datasets

- **15 total datasets**: combinations of 5 CNF formulas (300/500/1000/1500/1800 clauses) × 3 data sizes (100/1000/5000 examples)
- Each split into **train/validation/test** sets

- Binary classification: solutions to Boolean formulas → class 1, non-solutions → class 0
- All attributes are binary (0 or 1)
- Files already in your workspace: all_data

Four Classifiers to Evaluate

1. **Decision Tree** - tune criterion, max_depth, min_samples_split, min_samples_leaf
2. **Bagged DT** - tune n_estimators (10/20/50/100) and max_depth of base trees
3. **Random Forest** - tune n_estimators and max_features (sqrt, log2, fractions)
4. **AdaBoost** - tune n_estimators and base tree depth (1/3/5)

Workflow (Same for All 4)

1. **Grid search hyperparameter tuning** on validation set (no k-fold)
2. **Select best configuration** based on validation accuracy
3. **Retrain on combined train+validation** with best config
4. **Evaluate on test set** — report accuracy and F1 score

Deliverables

- Code with README (must run without errors)
- Report with:
  - Validation accuracy tables for hyperparameter tuning
  - Test accuracy/F1 tables (formatted like Table 2 in the spec)
  - Reflections on how hyperparameters affect bias/variance
  - Comparative analysis: which classifier is best, impact of data size, impact of formula complexity
- AI chat transcript + report

Key constraint: Don't mix datasets (e.g., don't train on `train_c500_d100` and test on `test_c500_d5000`).

Is this all clear? Ready to start building the experimental code?

User: Yeah let's start with steps. As like usual, don't code anything for me, just let me know what I need to do. Let's start with the first part, should I reate a sklearn.tree.DescionTreeClassifier.py?

GitHub Copilot: Good question - let me suggest a practical structure instead.

Rather than creating a file named after the sklearn class, I'd recommend organizing it **by workflow/functionality** for cleaner structure and to avoid repetition. Here's what I'd suggest:

Recommended Structure

1. `utils.py` (or `helpers.py`)
   - Load dataset function (train/valid/test from CSV)
   - Evaluation function (accuracy, F1 score)
   - Any other common utilities
2. `project2.py` (main orchestrator)
   - Runs experiments for all 4 classifiers
   - Loops through all 15 datasets
   - Calls functions from individual classifier modules
3. **Separate files for each classifier:**
   - `decision_tree.py` - handles hyperparameter tuning, training, evaluation for DT
   - `bagging.py` - handles workflow for Bagged DT
   - `random_forest.py` - handles workflow for RF
   - `adaboost.py` - handles workflow for AdaBoost

Alternative (More Compact)

Just **one main script** with functions for each classifier, if you prefer to keep it simpler.

My recommend: Start with `utils.py` to handle the common stuff (loading data, evaluation metrics), then create the individual classifier modules. This way each classifier's workflow is self-contained and you avoid code duplication.

Which approach appeals to you?

User: I'll go with what you recommend. Alright I'm working on util.py - what do I need?

GitHub Copilot: Looking at your project needs, here are the **essential functions** for `utils.py`:

Core Functions Needed

1. `load_dataset(filepath)`
   - Read CSV file
   - Split into features (X) and labels (y)
   - Return X, y
   - Handle the fact that last column is the class label
2. `get_dataset_combinations()`
   - Return all 15 dataset combinations: (clauses, data_size)
   - i.e., (300, 100), (300, 1000), (300, 5000), (500, 100), ... (1800, 5000)
   - Useful for looping in your main script
3. `load_train_valid_test(clauses, data_size, base_path)`
   - Given clauses and data_size, load all three datasets (train, valid, test)
   - Return (X_train, y_train), (X_valid, y_valid), (X_test, y_test)
   - Prevents mixing datasets accidentally
4. `evaluate_model(y_true, y_pred)`
   - Calculate accuracy
   - Calculate weighted F1 score
   - Return dict: `{'accuracy': ..., 'f1': ...}`

Optional but Nice-to-Have

- A function to save results to CSV (for collecting all your results later)
- A function to get a standardized filepath given dataset params

Start with those 4 core functions. Does that make sense? Want me to outline the function signatures/docstrings?

User: When I load dataset and return x,y should I do it as int or float?

GitHub Copilot: **Use `float`.**

Here's why:

1. **Standard in ML workflows** — Most scikit-learn examples and ML code uses float32/float64
2. **Future-proofing** — If you ever scale/normalize features later, float is expected
3. **No downside** — Your datasets are small enough that memory isn't a concern
4. **Tree classifiers don't care** — Decision trees handle both equally, but float is the convention

You can load with `dtype=float` in pandas:

```
df = pd.read_csv(filepath, dtype=float)
```

That's it — keep it simple and use float.

User: What does iterating an array like [:, :-1] vs [:, -1] mean

GitHub Copilot: **Array slicing notation** (NumPy/Pandas):

- `[:, :-1]` = **all rows, all columns EXCEPT the last**
  - `:` = all rows
  - `:-1` = up to (but not including) the last column
- `[:, -1]` = **all rows, ONLY the last column**
  - `:` = all rows

- `-1` = the last column specifically

In Your Context

Perfect for splitting features from labels:

```
X = df[:, :-1]  # All columns except last → features
y = df[:, -1]   # Last column only → class labels
```

This matches your project spec: "The last attribute corresponds to the class variable."

User: Does this mean that 100, 1000, 5000 for data_sizes? and 300,...,1800 are clauses?

Five CNF formulas were generated with 500 variables and varying numbers of clauses: 300, 500, 1000, 1500, and 1800 clauses (each clause containing exactly 3 literals). From each formula, 100, 1000, and 5000 positive (class=1) and negative (class=0) examples were sampled. Therefore there are a total of 15 different classification problems.

GitHub Copilot: **Exactly yes.**

From the spec:

- **Clauses**: 300, 500, 1000, 1500, 1800 (the CNF formula complexity)
- **Data sizes**: 100, 1000, 5000 (number of examples per dataset)

Filename breakdown:

- `train_c500_d100.csv` = 100 examples from the CNF formula with 500 clauses
- `test_c1800_d5000.csv` = 5000 examples from the CNF formula with 1800 clauses

Total: 5 clauses × 3 data sizes = 15 datasets

So for `get_dataset_combinations()`, you'd return:

```
[(300, 100), (300, 1000), (300, 5000),
 (500, 100), (500, 1000), (500, 5000),
 (1000, 100), (1000, 1000), (1000, 5000),
 (1500, 100), (1500, 1000), (1500, 5000),
 (1800, 100), (1800, 1000), (1800, 5000)]
```

Where each tuple is `(clauses, data_size)`.

User: How is my code? I worry about having some missing edge cases in the evaluate model

GitHub Copilot: Good instinct — there are **some issues** with edge cases and logic flow:

Problems

1. **Variable scope issue** — If `tp + fp == 0`, then `precision` and `recall` are never defined, but then `if precision + recall > 0:` tries to use them → **NameError**
2. **Accuracy always valid** — Accuracy calculation should **always** happen (move outside the if), since it's just correct predictions / total samples
3. **Division by zero edge cases**:
   - If no predicted positives: `tp + fp == 0` → can't calculate precision
   - If no actual positives: `tp + fn == 0` → can't calculate recall
   - These aren't explicitly handled
4. **Better approach** — Use **sklearn's built-in functions** instead:

   ```
   from sklearn.metrics import accuracy_score, f1_score

   accuracy = accuracy_score(y_true, y_pred)
   f1 = f1_score(y_true, y_pred, average='weighted')  # This is what project asks for
   ```

Quick Fix

Replace `evaluate_model` with:

```
from sklearn.metrics import accuracy_score, f1_score

def evaluate_model(y_pred, y_true):
    accuracy = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average='weighted')
    return {'accuracy': accuracy, 'f1_score': f1}
```

Simpler, cleaner, handles edge cases automatically. The project specifically asks for "weighted F1 score" anyway.

User: Is that what you get from reading the original pdf? Like that I should sklearn for everything?

GitHub Copilot: Good catch — no, not explicitly.

The spec says: **"You will use scikit-learn"** at the top, and it specifies the classifiers to use (DecisionTreeClassifier, BaggingClassifier, etc.). But for **evaluation**, it just says:

> "report: Classification accuracy, F1 score (weighted average)"

It doesn't mandate using `sklearn.metrics`. Your manual calculation approach is **valid**.

However, my recommendation to use `sklearn.metrics` was based on:

1. **Best practice** — sklearn's functions handle edge cases robustly
2. **Alignment** — Since you're already using sklearn for classifiers, using their metrics is natural
3. **Reliability** — Your manual calculation had potential bugs (as we noted)

You could go either way. If you want to stick with your manual approach, just **fix the edge cases** (handle division by zero). Or switch to sklearn's functions for simplicity.

Your call — I shouldn't have presented it as a requirement when it wasn't. What do you prefer?

User: I plan on trying to do anything that I can implement myself to do it myself. I will use the sklearn.tree features but personally it's a very useful thing to be able to understand the fundamentals. First principles, second order thinking, systems thinking is how I want you to approach me.

Anyways, is my utils.py fine? Next up is decision tree

GitHub Copilot: **utils.py looks good overall**, but there's **one bug** in `load_train_valid_test()`:

The Issue

Your filepath format is wrong. Looking at your data folder, files are named:

- `train_c500_d100.csv` (not `500_100_train.csv`)

So this line:

```
train_path = os.path.join(base_path, f'{clauses}_{data_size}_train.csv')
```

Should be:

```
train_path = os.path.join(base_path, f'train_c{clauses}_d{data_size}.csv')
```

Fix that and you're good to move forward.

Now for `decision_tree.py`

Before you code, think about **what this module needs to do** (first principles):

1. **Grid search hyperparameter tuning** — You need to:
   - Define a search space for: criterion, max_depth, min_samples_split, min_samples_leaf
   - Try all combinations

    - Train a model on train set, evaluate on validation set
    - Track which combo performed best
2. **Model selection** — Pick the best hyperparameters based on validation accuracy
3. **Retraining** — Combine train + validation, retrain with best params
4. **Final evaluation** — Evaluate on test set, return accuracy + F1

What's the key question: **How do you systematically explore all combinations without manually writing nested loops?** What tool/approach would handle that elegantly?

User: Wait since I'm importing from sklearn.tree, what's the point of decision_tree.py? I don't even know what to put for it since everything can be done in a main no? There's no need for classes right?

GitHub Copilot: **You're right to question it.** You don't *need* classes here.

Since sklearn is already doing the heavy lifting, a **class wrapper adds unnecessary complexity**. You just need **functions**.

Better approach:

```
# decision_tree.py - just functions, no classes

def tune_and_evaluate_decision_tree(X_train, y_train, X_valid, y_valid, X_test, y_test):
    # 1. Define hyperparameter search space
    # 2. Grid search over combinations
    # 3. Track best validation accuracy
    # 4. Combine train + valid, retrain
    # 5. Evaluate on test set
    # 6. Return results dict
    pass
```

Then in `main.py`:

```
results = decision_tree.tune_and_evaluate_decision_tree(X_train, y_train, X_valid, y_valid, X_test, y_test)
```

Same pattern for `bagging.py`, `random_forest.py`, `adaboost.py` — each has one function that does the full workflow.

Why separate functions instead of all in main?

- **DRY principle** — Each classifier follows identical workflow (grid search → select → retrain → eval). Functions avoid repetition.
- **Readability** — main.py stays clean: just loop through datasets, call the function, collect results
- **Maintainability** — Change grid search logic once instead of 4 times

Does that make sense? Want to start outlining what needs to happen *inside* that decision tree function?

User: Do I need a random state and what is it? And what is the point of max_depth = max_depth?

GitHub Copilot: Good questions — both fundamental concepts:

`random_state`

A **seed for reproducibility**. Decision trees use randomness for tie-breaking when multiple features have equal information gain.

Without it:

```
clf1 = DecisionTreeClassifier(criterion='gini')
clf1.fit(X, y)  # Might pick feature A or B randomly
# Later...
clf2 = DecisionTreeClassifier(criterion='gini')
clf2.fit(X, y)  # Might pick differently - different tree!
```

With `random_state=42`:

```
clf1 = DecisionTreeClassifier(criterion='gini', random_state=42)
clf1.fit(X, y)  # Always picks the same way
# Later...
```

```
clf2 = DecisionTreeClassifier(criterion='gini', random_state=42)
clf2.fit(X, y)  # Same tree every time
```

For your project: YES, use it. Makes results reproducible so you can explain exactly what happened.

`max_depth = max_depth`

This is just **parameter assignment syntax**:

```
clf = DecisionTreeClassifier(max_depth=max_depth)
```

- **Left side** (`max_depth=`): formal parameter name that sklearn expects
- **Right side** (`max_depth`): the variable you're passing in from your grid search loop

Example in context:

```
for max_depth in [5, 10, None]:  # grid search loop
    clf = DecisionTreeClassifier(max_depth=max_depth)  # passing the value
    # now clf has max_depth=5, then 10, then None
```

Does that clarify both?

User: Alright how is all of this now?

GitHub Copilot: **Overall structure is solid**, but there are **a few bugs to fix**:

Issues

1. **Typo in main.py**

   ```
   import numyp as np  # ❌ should be numpy
   ```

2. **Tuple unpacking error in decision_tree.py**

   ```
   accuracy, f1 = utils.evaluate_model(y_pred, y_test)  # ❌ Won't work
   ```

   Your `evaluate_model()` returns a **dict**, not a tuple. Should be:

   ```
   results = utils.evaluate_model(y_pred, y_test)
   accuracy = results['accuracy']
   f1 = results['f1_score']
   ```

3. **Missing criterion in grid search**
   The spec says: "Explore combinations of the following hyperparameters: Splitting criterion (e.g., **gini, entropy**)"

   You need to add to your grid:

   ```
   criterions = ['gini', 'entropy']
   ```

   And loop through it.

4. **Missing main() call**
   At bottom of main.py, add:

   ```
   if __name__ == '__main__':
       main()
   ```

5. **Still need to fix utils.py filepath** (from earlier):

   ```
   train_path = os.path.join(base_path, f'train_c{clauses}_d{data_size}.csv')
   ```

Once those are fixed, test it to see if it actually runs without errors.

User: Ok I fixed them all. #codebase

GitHub Copilot: Searched codebase for "Ok I fixed them all. #codebase ", 13 results

Good. Now **test it** to make sure there are no runtime errors.

From terminal in the proj2 directory:

```
python main.py
```

This will run decision tree on all 15 datasets. It might take a bit since it's testing 4 × 3 × 3 × 2 = 72 models per dataset.

Once it runs successfully (or if you hit errors), let me know. Then we'll move to the next three classifiers (`bagging.py` , `random_forest.py` , `adaboost.py`) using the same pattern.

User: How come I'm getting this error?

.venv) C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 5, in
import utils
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\utils.py", line 30, in
import numpy as np
ModuleNotFoundError: No module named 'numpy'

GitHub Copilot: You need to install the required packages in your virtual environment.

Run:

```
pip install numpy scikit-learn pandas
```

This will install:

- `numpy` — for array operations
- `scikit-learn` — for the classifiers
- `pandas` — (optional but good to have for CSV operations, though you're using csv module)

After that, try `python main.py` again.

User: How do I get out of .venv?

GitHub Copilot: Just type:

```
deactivate
```

Then your prompt will return to normal (no more `(.venv)` prefix).

But wait — you probably *want* to stay in the .venv to install packages and run your code. Are you trying to get out for a reason?

User: Hm? Am I doing the base path wrong?

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 22, in
main()
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 14, in main
x_train, y_train, x_valid, y_valid, x_test, y_test = utils.load_train_valid_test(clauses, data_size, base_path)
^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\utils.py", line 56, in load_train_valid_test
x_train, y_train = load_dataset(train_path)
^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\utils.py", line 33, in load_dataset
with open(filepath, 'r') as f:
^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'project2_data\all_data\train_c300_d100_train.csv'

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>

GitHub Copilot: Look at the error — the filename is:

```
train_c300_d100_train.csv
```

But it should be:

```
train_c300_d100.csv
```

You have `train` appearing **twice**. Check your `utils.py` `load_train_valid_test()` function. It should be:

```
def load_train_valid_test(clauses, data_size, base_path):
    train_path = os.path.join(base_path, f'train_c{clauses}_d{data_size}.csv')
    valid_path = os.path.join(base_path, f'valid_c{clauses}_d{data_size}.csv')
    test_path = os.path.join(base_path, f'test_c{clauses}_d{data_size}.csv')
```

Does your current code have that exact pattern, or did it accidentally keep something from the old version?

User: Alright. It all worked. Take a look at the results. They seem good.

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 10, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.6533, Total Models Evaluated: 72
Test Accuracy: 0.6281, Test F1 Score: 0.6263

Processing dataset with 300 clauses and 1000 data points...
Best hyperparameters: {'max_depth': 5, 'min_samples_split': 2, 'min_samples_leaf': 1, 'criterion': 'entropy'}, Validation Accuracy: 0.6493, Total Models Evaluated: 72
Test Accuracy: 0.6723, Test F1 Score: 0.7093

Processing dataset with 300 clauses and 5000 data points...
Best hyperparameters: {'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy:

0.7382, Total Models Evaluated: 72
Test Accuracy: 0.7803, Test F1 Score: 0.7884

Processing dataset with 500 clauses and 100 data points...
Best hyperparameters: {'max_depth': 5, 'min_samples_split': 2, 'min_samples_leaf': 4, 'criterion': 'entropy'}, Validation Accuracy: 0.6482, Total Models Evaluated: 72
Test Accuracy: 0.6734, Test F1 Score: 0.6948

Processing dataset with 500 clauses and 1000 data points...
Best hyperparameters: {'max_depth': 5, 'min_samples_split': 2, 'min_samples_leaf': 1, 'criterion': 'entropy'}, Validation Accuracy: 0.6938, Total Models Evaluated: 72
Test Accuracy: 0.6818, Test F1 Score: 0.6867

Processing dataset with 500 clauses and 5000 data points...
Best hyperparameters: {'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.7566, Total Models Evaluated: 72
Test Accuracy: 0.7886, Test F1 Score: 0.7981

Processing dataset with 1000 clauses and 100 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.7588, Total Models Evaluated: 72
Test Accuracy: 0.7337, Test F1 Score: 0.7440

Processing dataset with 1000 clauses and 1000 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 4, 'criterion': 'entropy'}, Validation Accuracy: 0.8064, Total Models Evaluated: 72
Test Accuracy: 0.7889, Test F1 Score: 0.7965

Processing dataset with 1000 clauses and 5000 data points...
Best hyperparameters: {'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.8435, Total Models Evaluated: 72
Test Accuracy: 0.8601, Test F1 Score: 0.8652

Processing dataset with 1500 clauses and 100 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 10, 'min_samples_leaf': 4, 'criterion': 'entropy'}, Validation Accuracy: 0.8744, Total Models Evaluated: 72
Test Accuracy: 0.8794, Test F1 Score: 0.8800

Processing dataset with 1500 clauses and 1000 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.9165, Total Models Evaluated: 72
Test Accuracy: 0.9270, Test F1 Score: 0.9279

Processing dataset with 1500 clauses and 5000 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 10, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.9482, Total Models Evaluated: 72
Test Accuracy: 0.9565, Test F1 Score: 0.9566

Processing dataset with 1800 clauses and 100 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.9749, Total Models Evaluated: 72
Test Accuracy: 0.9146, Test F1 Score: 0.9187

Processing dataset with 1800 clauses and 1000 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'criterion': 'entropy'}, Validation Accuracy: 0.9710, Total Models Evaluated: 72
Test Accuracy: 0.9780, Test F1 Score: 0.9781

Processing dataset with 1800 clauses and 5000 data points...
Best hyperparameters: {'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 4, 'criterion': 'entropy'}, Validation Accuracy: 0.9842, Total Models Evaluated: 72
Test Accuracy: 0.9873, Test F1 Score: 0.9873

GitHub Copilot: **Results look solid.** Some good patterns to notice:

1. **Entropy always wins** — Every single best config uses `criterion='entropy'`. Gini never selected. Interesting!

2. **Performance trends are clear:**
   - **More data = better** (c300 d100: 0.6281 → c300 d5000: 0.7803)
   - **More clauses = better** (c300: ~0.63-0.78 → c1800: ~0.91-0.99)
   - Best case (c1800 d5000): 98.73% accuracy
3. **max_depth isn't always None** — Despite the project worrying about "fully grown trees overfitting," constrained depths (5, 10) sometimes win, especially on smaller datasets
4. **Regularization helps on small data** — When d=100, the algorithm often picks higher `min_samples_split` or `min_samples_leaf` values

---

Next Steps

You now have **Decision Tree working**. Three more classifiers to go:

1. `bagging.py` — BaggingClassifier with base DecisionTree
2. `random_forest.py` — RandomForestClassifier
3. `adaboost.py` — AdaBoostClassifier

Then create **comparison tables** across all four.

Ready to tackle **Bagging next**? Similar structure — grid search, but now tuning `n_estimators` and `max_depth` of base trees.

User: Explain the difference between gini and entropy. Like how it works

GitHub Copilot: **Both measure the same thing conceptually: how "mixed" or "pure" a node is.**

The tree splits data by asking: "Which feature split makes the groups more pure?" It uses Gini or Entropy to measure this.

Entropy (Information Theory)

Formula: $_{i}\ _{i}$

Where $_{i}$ = proportion of class i

Example — Binary classification:

- **Pure node** (all class 1): $1\ \ 1$ entropy
- **Mixed node** (50% class 0, 50% class 1): 1 entropy
- **Mostly class 0** (90% class 0, 10% class 1): $1\ \ 1$ entropy

Higher entropy = more disorder/uncertainty

Gini Impurity (Probability-based)

Formula: $1\ _{i}$

Same examples:

- **Pure node**: $1\ \ 1$ gini
- **Mixed node** (50-50): 1 gini
- **90-10**: $1\ \ 1\ \ 1$ gini

Higher gini = more impurity

The Difference

- **Conceptually**: Entropy is from information theory (log-based), Gini is probability-based
- **Practically**: Results are usually **very similar**
- **Computationally**: Gini is slightly faster (no logarithm)

Both work by: **Pick the split that maximizes the purity gain.**

In your results, **entropy won every time** — probably just a slight edge on this dataset, but they're close enough that either would work fine.

User: And one thing, how come we have to put if **name** == "**main**":
main() at the end of main.py?

GitHub Copilot: Good question — it's about **preventing accidental code execution**.

When you write this in `main.py`:

```
main()
```

Two scenarios happen:

Scenario 1: Running directly

```
python main.py
```

→ `main()` executes ✓ (what you want)

Scenario 2: Importing it

```
# some_other_file.py
import main
```

→ `main()` executes immediately ✗ (probably not what you want)

With the guard:

```python
if __name__ == "__main__":
    main()
```

- `__name__` = `"__main__"` when you run it directly → executes
- `__name__` = `"main"` when imported → doesn't execute

In practice for your project: It doesn't matter much since you're not importing `main.py` anywhere. But it's **Python best practice** to always use this pattern. Makes code reusable and prevents surprises.

Think of it as: "Only run this if I'm the main script being executed, not if someone imported me."

User: Somehow I messed up my code entirely.

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 37, in
main()
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 29, in main
best_params_bagging, best_accuracy_bagging, total_models_bagging = bagging.tune_evaluate_bagging(x_train, y_train, x_valid, y_valid)
^^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\bagging.py", line 37, in tune_evaluate_bagging
model = BaggingClassifier(base_estimator=base_estimator, n_estimators=n_estimators, random_state=42)
^^^
TypeError: BaggingClassifier.**init**() got an unexpected keyword argument 'base_estimator'

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>

GitHub Copilot: This is a **scikit-learn version issue**. In newer versions of sklearn, the parameter was renamed.

Change:

```
base_estimator=base_estimator
```

To:

```
estimator=base_estimator
```

In your `bagging.py`, find the line with `BaggingClassifier(base_estimator=...` and replace `base_estimator=` with `estimator=`.

That should fix it.

User: Alright that fixed it. Take a look at my main.py

I was going to set up some flags so I can skip processing decision tree every single time I'm testing out bagging.py but I think I ended up just breaking it

GitHub Copilot: Good idea with the flags, but **one bug**: you're only creating `x_train_valid` and `y_train_valid` inside the `if DTC:` block. When `DTC = False`, those variables don't exist, so the bagging section crashes.

Fix: Move the concatenation **outside** the conditionals:

```
def main():
    base_path = 'project2_data\\all_data\\'
    dataset_combinations = utils.get_dataset_combinations()
    for clauses, data_size in dataset_combinations:
        print(f"Processing dataset with {clauses} clauses and {data_size} data points...")
        x_train, y_train, x_valid, y_valid, x_test, y_test = utils.load_train_valid_test(clauses, data_size,
base_path)

        # Always create combined set
        x_train_valid = np.concatenate((x_train, x_valid), axis=0)
        y_train_valid = np.concatenate((y_train, y_valid), axis=0)
```

```
        # decision tree classifier
        if DTC:
            best_params, best_accuracy, total_models = decision_tree.tune_evaluate_decision_tree(x_train,
y_train, x_valid, y_valid)
            # ...rest of DTC code

        # bagged decision tree classifiers
        if Bagged:
            best_params_bagging, best_accuracy_bagging, total_models_bagging =
bagging.tune_evaluate_bagging(x_train, y_train, x_valid, y_valid)
            # ...rest of Bagging code using x_train_valid, y_train_valid
```

Now the combined set is always available regardless of which flags are True/False.

User: And just wondering, are flags typically all caps?

GitHub Copilot: **Not typically.** Python conventions:

- **Constants** (rarely change): `ALL_CAPS` — e.g., `MAX_DEPTH = 5`
- **Variables/Flags** (can change): `lowercase_with_underscores` — e.g., `use_dtc = False`

Your current flags:

```
DTC = False
Bagged = True
```

Should be:

```
use_dtc = False
use_bagging = True
```

Or if you want to emphasize they're config/constants:

```
ENABLE_DTC = False
ENABLE_BAGGING = True
```

The all-caps `DTC` and `Bagged` look like class names or constants, which is confusing. Lowercase with descriptive names is clearer for flags.

User: Hmm... It took an insanely long time to process so I stopped it. I think there's an error but what do you think? I stopped it already.

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 50, 'max_depth': 5}, Validation Accuracy: 0.7286, Total Models Evaluated: 12
Bagging Test Accuracy: 0.7739, Bagging Test F1 Score: 0.7867

Processing dataset with 300 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.8689, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8904, Bagging Test F1 Score: 0.8922

Processing dataset with 300 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.8985, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9185, Bagging Test F1 Score: 0.9230

Processing dataset with 500 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': 5}, Validation Accuracy: 0.7990, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8543, Bagging Test F1 Score: 0.8599

Processing dataset with 500 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': 10}, Validation Accuracy: 0.8784, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8714, Bagging Test F1 Score: 0.8705

Processing dataset with 500 clauses and 5000 data points...
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 38, in
main()
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 30, in main
best_params_bagging, best_accuracy_bagging, total_models_bagging = bagging.tune_evaluate_bagging(x_train, y_train, x_valid, y_valid)
^^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\bagging.py", line 39, in tune_evaluate_bagging
y_pred = model.predict(x_valid)
^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\ensemble_bagging.py", line 903, in predict
predicted_probabilitiy = self.predict_proba(X)
^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\ensemble_bagging.py", line 942, in predict_proba
all_proba = Parallel(
^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\parallel.py", line 77, in **call**
return super().**call**(iterable_with_config)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\parallel.py", line 1863, in **call**
return output if self.return_generator else list(output)
^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\parallel.py", line 1792, in _get_sequential_output
res = func(*args, **kwargs)*
^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\parallel.py", line 139, in

call
*return self.function(*args, **kwargs)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\ensemble_bagging.py", line 212, in _parallel_predict_proba
proba_estimator = estimator.predict_proba(X[:, features])
^^^
File "C:\Users\DragonFire\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\tree_classes.py", line 1032, in predict_proba
def predict_proba(self, X, check_input=True):

KeyboardInterrupt
^C
C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>

GitHub Copilot: **Not an error** — a `KeyboardInterrupt` just means you stopped it with Ctrl+C.

The real issue: performance. Look at what it reached:

Dataset `500 clauses and 5000 data points` — this is:

- **5000 examples** (large)
- **500 features** (large)
- **100 estimators** in the ensemble (trying to predict with 100 trees)

On that dataset, `predict_proba()` is **compute-intensive**. With 100 bagged trees × 5000 samples, it's slow.

Why did smaller datasets finish quickly? They had fewer examples (100-1000) or fewer clauses (300-500).

Options

1. **Just wait it out** — It will likely finish, just takes time
2. **Reduce grid search space** — Use fewer `n_estimators` values (e.g., just `[10, 50, 100]` instead of more)
3. **Optimize parallel processing** — Add `n_jobs=-1` to BaggingClassifier to use all CPU cores
4. **Skip ahead** — Disable bagging, implement Random Forest and AdaBoost first, then come back

What do you prefer?

User: Ok yes. Let's do 3. Parallel processing, so for all calls to BaggingClassifier n_jobs-=1?

GitHub Copilot: Yes, add `n_jobs=-1` to both BaggingClassifier calls in your bagging.py:

1. In `tune_evaluate_bagging()`:

```
model = BaggingClassifier(estimator=base_estimator, n_estimators=n_estimators, random_state=42, n_jobs=-1)
```

2. In `retrain_evaluate_bagging()`:

```
model = BaggingClassifier(estimator=base_estimator, n_estimators=best_params['n_estimators'],
random_state=42, n_jobs=-1)
```

`n_jobs=-1` tells it to use **all CPU cores** for parallel training. This should speed things up significantly on larger datasets.

User: Alright how does this look?

Processing dataset with 300 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 50, 'max_depth': 5}, Validation Accuracy: 0.7286, Total Models Evaluated: 12
Bagging Test Accuracy: 0.7739, Bagging Test F1 Score: 0.7867

Processing dataset with 300 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.8689, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8904, Bagging Test F1 Score: 0.8922

Processing dataset with 300 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.8985, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9185, Bagging Test F1 Score: 0.9230

Processing dataset with 500 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': 5}, Validation Accuracy: 0.7990, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8543, Bagging Test F1 Score: 0.8599

Processing dataset with 500 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': 10}, Validation Accuracy: 0.8784, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8714, Bagging Test F1 Score: 0.8705

Processing dataset with 500 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9129, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9432, Bagging Test F1 Score: 0.9437

Processing dataset with 1000 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.8593, Total Models Evaluated: 12
Bagging Test Accuracy: 0.8894, Bagging Test F1 Score: 0.8866

Processing dataset with 1000 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9460, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9430, Bagging Test F1 Score: 0.9436

Processing dataset with 1000 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9523, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9636, Bagging Test F1 Score: 0.9635

Processing dataset with 1500 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 50, 'max_depth': 5}, Validation Accuracy: 0.9799, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9698, Bagging Test F1 Score: 0.9706

Processing dataset with 1500 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9795, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9825, Bagging Test F1 Score: 0.9824

Processing dataset with 1500 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9889, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9904, Bagging Test F1 Score: 0.9904

Processing dataset with 1800 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 10, 'max_depth': None}, Validation Accuracy: 1.0000, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9698, Bagging Test F1 Score: 0.9700

Processing dataset with 1800 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9910, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9925, Bagging Test F1 Score: 0.9925

Processing dataset with 1800 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9966, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9976, Bagging Test F1 Score: 0.9976

GitHub Copilot: **Bagging results look excellent.** Some key observations:

1. **Performance boost over Decision Tree** — Bagging consistently outperforms single DT across all datasets. Example:

- DT c300 d100: 0.6281 accuracy
- Bagging c300 d100: 0.7739 accuracy

This is the variance reduction benefit of ensembles at work.

2. **Pattern:** `n_estimators=100` **dominates** — Almost every best config uses 100 estimators. More trees = more vote averaging = lower variance.
3. `max_depth=None` **preferred** — Most wins with fully grown trees. Interesting because individual bagged trees can overfit, but the ensemble averages it out.
4. **One curiosity**: c1800 d100 has validation accuracy **1.0** but test accuracy 0.9698. This suggests the validation set might just be particularly easy, or there's slight overfitting. But it's not alarming — test performance is still strong.
5. **Speed**: Much faster with `n_jobs=-1` ✓

Next: Random Forest & AdaBoost

Ready to move to `random_forest.py` and `adaboost.py`? Same structure, different hyperparameter grids:

- **RF**: tune `n_estimators` and `max_features`
- **AdaBoost**: tune `n_estimators` and base tree depth

Want to start?

User: Should I have max_depth option on adaboost to have none or no? Pdf says 1,3,5

GitHub Copilot: **No — stick with 1, 3, 5 only.**

Here's the conceptual difference:

Bagging: Works well with strong learners (fully grown trees). More powerful trees = better ensemble.

AdaBoost: Requires *weak learners*. The algorithm works by:

1. Train a weak learner
2. Increase weights on misclassified examples
3. Train next weak learner on harder examples
4. Repeat, sequentially correcting mistakes

If you use `max_depth=None` (fully grown trees), they become **strong learners**, not weak ones. This breaks the AdaBoost algorithm's logic — you're no longer correcting mistakes iteratively; you're just combining already-perfect trees.

So the spec is intentional — only try depths 1, 3, 5 for AdaBoost. No None.

That's a good distinction to understand: **AdaBoost needs weak base learners to function properly.**

User: Do I need to import decisiontreeclassifier for adaboost too?

```
        base_estimator = DecisionTreeClassifier(max_depth=max_depth, random_state=42)
```

GitHub Copilot: Yes. Add this import at the top of `adaboost.py`:

```
from sklearn.tree import DecisionTreeClassifier
```

So your imports look like:

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
import numpy as np
import utils
```

AdaBoost needs to know what a DecisionTreeClassifier is to use it as the base estimator.

User: Hmm... it's the exact same code as the previous 3 times, just changed a little bit. How come base estimator bugs out for adaboost?

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 54, in
main()
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 48, in main
best_params_boost, best_accuracy_boost, total_models_boost = adaboost.tune_evaluate_adaboost(x_train, y_train, x_valid, y_valid)
^^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\adaboost.py", line 29, in tune_evaluate_adaboost
model = AdaBoostClassifier(base_estimator=base_estimator, n_estimators=n_estimators, random_state=42)
^^^
TypeError: AdaBoostClassifier.**init**() got an unexpected keyword argument 'base_estimator'

GitHub Copilot: Same issue as Bagging — newer sklearn renamed the parameter.

Change:

```
base_estimator=base_estimator
```

To:

```
estimator=base_estimator
```

In both places in your `adaboost.py`:

1. `tune_evaluate_adaboost()` — line with `AdaBoostClassifier(...)`
2. `retrain_evaluate_adaboost()` — same change

AdaBoost, Bagging, RandomForest all had this parameter renamed in newer sklearn versions.

User: No jobs allowed?

C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2>python main.py
Processing dataset with 300 clauses and 100 data points...
Traceback (most recent call last):
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 54, in
main()
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\main.py", line 48, in main
best_params_boost, best_accuracy_boost, total_models_boost = adaboost.tune_evaluate_adaboost(x_train, y_train, x_valid, y_valid)
^^^
File "C:\Users\DragonFire\Desktop\Coding\Machine Learning\proj2\adaboost.py", line 29, in tune_evaluate_adaboost
model = AdaBoostClassifier(estimator=base_estimator, n_estimators=n_estimators, random_state=42, n_jobs=-1)
^^^
TypeError: AdaBoostClassifier.**init**() got an unexpected keyword argument 'n_jobs'

GitHub Copilot: **AdaBoost doesn't support `n_jobs`** — it's inherently sequential. Each boosting round depends on the previous one, so you can't parallelize it like Bagging.

Remove `n_jobs=-1` from both AdaBoost calls in `adaboost.py`:

```
model = AdaBoostClassifier(estimator=base_estimator, n_estimators=n_estimators, random_state=42)
```

(No `n_jobs` parameter)

Why? Bagging and Random Forest can train trees independently (in parallel). AdaBoost must train sequentially — each tree focuses on examples the previous tree got wrong. That's how it works.

User: Take a look at this table. I want you to help with formatting all the data into a table. You don't have to code it, but just structure it. I'll make a table in obsidian and print it out as a report.

GitHub Copilot: **Table Structure for Your Report:**

I'll organize it as two tables (one for Accuracy, one for F1 Score) to match the project spec.

Table 1: Test Set Classification Accuracy

Dataset	DecisionTree	Bagging	RandomForest	AdaBoost
c300_d100	0.6281	0.7739	0.7186	0.8241
c300_d1000	0.6723	0.8904	0.8839	0.9380
c300_d5000	0.7803	0.9185	0.9091	0.9845
c500_d100	0.6734	0.8543	0.8543	0.8693
c500_d1000	0.6818	0.8714	0.9410	0.9770
c500_d5000	0.7886	0.9432	0.9566	0.9906
c1000_d100	0.7337	0.8894	0.9749	0.9548
c1000_d1000	0.7889	0.9430	0.9910	0.9910
c1000_d5000	0.8601	0.9636	0.9949	0.9987
c1500_d100	0.8794	0.9698	1.0000	0.9899
c1500_d1000	0.9270	0.9825	1.0000	1.0000
c1500_d5000	0.9565	0.9904	0.9999	0.9999
c1800_d100	0.9146	0.9698	0.9950	1.0000
c1800_d1000	0.9780	0.9925	0.9995	0.9995
c1800_d5000	0.9873	0.9976	0.9999	1.0000

Table 2: Test Set F1 Score (Weighted Average)

Dataset	DecisionTree	Bagging	RandomForest	AdaBoost
c300_d100	0.6263	0.7867	0.7255	0.8309
c300_d1000	0.7093	0.8922	0.8866	0.9386
c300_d5000	0.7884	0.9230	0.9118	0.9846
c500_d100	0.6948	0.8599	0.8543	0.8725
c500_d1000	0.6867	0.8705	0.9413	0.9770
c500_d5000	0.7981	0.9437	0.9573	0.9906
c1000_d100	0.7440	0.8866	0.9746	0.9538
c1000_d1000	0.7965	0.9436	0.9910	0.9910
c1000_d5000	0.8652	0.9635	0.9949	0.9987
c1500_d100	0.8800	0.9706	1.0000	0.9900
c1500_d1000	0.9279	0.9824	1.0000	1.0000
c1500_d5000	0.9566	0.9904	0.9999	0.9999
c1800_d100	0.9187	0.9700	0.9949	1.0000
c1800_d1000	0.9781	0.9925	0.9995	0.9995
c1800_d5000	0.9873	0.9976	0.9999	1.0000

Quick observations to guide your analysis:

- **Random Forest wins most datasets** (especially high-complexity ones: c1500, c1800)
- **AdaBoost competitive on medium-sized data**
- **DecisionTree worst** (baseline)
- **Bagging middle ground** between DT and ensembles

Copy these tables directly into Obsidian and format as markdown tables.

User: Now I need this table. I was thinking of just going to bagging.py and creating a new print function or staying in main... wait no. I think it's best to just have an extra parameter in the bagging functions that set default to false, but when enabled can allow me to print different values of n estimators and max depth

Table 1: Validation Accuracy of Bagged Decision Trees for Different Values of n estimators and
max depth
Dataset
10 20 50 100
max depth max depth max depth max depth
5 10 None 5 10 None 5 10 None 5 10 None
c300 d100 – – – – – – – – – – – –
c300 d1000 – – – – – – – – – – – –
c300 d5000 – – – – – – – – – – – –
c500 d100 – – – – – – – – – – – –
c500 d1000 – – – – – – – – – – – –
c500 d5000 – – – – – – – – – – – –
c1000 d100 – – – – – – – – – – – –
c1000 d1000 – – – – – – – – – – – –
c1000 d5000 – – – – – – – – – – – –
c1500 d100 – – – – – – – – – – – –
c1500 d1000 – – – – – – – – – – – –
c1500 d5000 – – – – – – – – – – – –
c1800 d100 – – – – – – – – – – – –
c1800 d1000 – – – – – – – – – – – –
c1800 d5000 – – – – – – – – – – – –

GitHub Copilot: Good thinking. A parameter approach works, but here's a **cleaner option:**

Instead of a flag, have `tune_evaluate_bagging()` **return the full results grid** in addition to the best params. Then in `main.py`, you can conditionally print it.

In bagging.py:

```
def tune_evaluate_bagging(x_train, y_train, x_valid, y_valid):
    # ... existing code ...
    results_grid = {}  # Track all results: {(n_est, max_depth): accuracy}

    for n_estimators in n_estimators_options:
        for max_depth in max_depth_options:
            # ... train and evaluate ...
            results_grid[(n_estimators, max_depth)] = accuracy

    return best_params, best_accuracy, total_models, results_grid
```

In main.py:

```
best_params, best_accuracy, total_models, results_grid = bagging.tune_evaluate_bagging(...)

if print_table:  # Toggle when needed
    print_bagging_table(results_grid, clauses, data_size)
```

Then create a simple helper function to format and print the table.

Why this is better:

- Doesn't pollute your function signatures with flags
- Separates data collection from presentation
- Easy to toggle on/off without code changes
- Reusable for all classifiers

Want me to outline what that helper function would look like?

User: Now that we've gone through all of them. What do you think is best and why? Like in my opinion, I notice that adaboost definitely has the best results, bagging vs random forest is similar, and normal decision trees are subpar.

Let's talk about the mechanisms and why

GitHub Copilot: **Excellent observation.** Let me break down the mechanisms:

Why AdaBoost Wins

The Core Difference: Sequential vs. Parallel

AdaBoost is sequential:

1. Train weak learner on original data
2. Identify examples it got wrong
3. Reweight those examples heavier
4. Train next learner on harder examples
5. Repeat — each learner specializes on what previous ones missed

Random Forest & Bagging are parallel:

- Train all trees independently on bootstrap samples
- Average their votes

Why This Matters for Boolean Functions

Boolean CNF formulas are **complex, non-linear boundaries** with many interacting clauses.

AdaBoost's advantage:

- Tree 1 learns some clauses
- Tree 2 focuses on examples Tree 1 missed (other clauses)

- Tree 3 fixes Tree 2's mistakes
- This **sequential refinement** directly addresses the structured complexity

Forced weak learners (max_depth=1,3,5):

- Each tree can't be too strong (can't capture everything alone)
- Forces *specialization* — each tree must focus on specific parts
- This diversity compounds benefits

Why RF/Bagging are OK but Not Best

Random Forest uses **feature subsampling** for diversity:

- Tree 1 considers features {1,2,3,50,100}
- Tree 2 considers features {5,10,20,75}
- Averaging reduces variance

But **all trees see the same examples equally.** For Boolean functions where some clauses are trickier than others, RF doesn't adapt — it treats (X1 ∨ ¬X2) the same as (¬X50 ∨ X2000).

AdaBoost says: "That clause is hard. Everyone focus on it."

Why Single Decision Tree Fails

Greedy splitting. Tree tries to maximize information gain at each node independently, but CNF clauses are interdependent. One tree can't navigate all the interactions, so it overfits locally or underfits globally.

Bottom line: AdaBoost's **sequential adaptation** matches the **structured complexity** of Boolean formulas better than parallel averaging. This is why it consistently wins — especially on high-complexity datasets (c1500, c1800).

Does this align with what you're seeing in the data?

User: Now I need you to help me make this table. Table 1 structured like this. Just organize the information, I'll make the table again.

Starting at line 255 of the personal.md is where you should look.

Dataset
10 20 50 100
max depth max depth max depth max depth
5 10 None 5 10 None 5 10 None 5 10 None
c300 d100 – – – – – – – – – – – –
c300 d1000 – – – – – – – – – – – –
c300 d5000 – – – – – – – – – – – –
c500 d100 – – – – – – – – – – – –
c500 d1000 – – – – – – – – – – – –
c500 d5000 – – – – – – – – – – – –
c1000 d100 – – – – – – – – – – – –
c1000 d1000 – – – – – – – – – – – –
c1000 d5000 – – – – – – – – – – – –
c1500 d100 – – – – – – – – – – – –
c1500 d1000 – – – – – – – – – – – –
c1500 d5000 – – – – – – – – – – – –
c1800 d100 – – – – – – – – – – – –
c1800 d1000 – – – – – – – – – – – –
c1800 d5000 – – – – – – – – – – – –

GitHub Copilot: Here's the organized data from your output extracted into the table format:

Validation Accuracy of Bagged Decision Trees

Dataset	**10**			**20**			**50**			**100**		
max_depth	5	10	None	5	10	None	5	10	None	5	10	None
c300_d100	0.6382	0.6231	0.6231	0.6834	0.6633	0.6633	0.7286	0.7035	0.7035	0.6884	0.6834	0.6834
c300_d1000	0.8074	0.7589	0.7589	0.8289	0.7849	0.7819	0.8519	0.8464	0.8449	0.8534	0.8674	0.8689
c300_d5000	0.7877	0.8408	0.8161	0.7823	0.8595	0.8564	0.7938	0.8849	0.8864	0.7920	0.8943	0.8985
c500_d100	0.6281	0.5980	0.5980	0.6734	0.6583	0.6583	0.7387	0.7186	0.7186	0.7990	0.7889	0.7889
c500_d1000	0.7989	0.7739	0.7739	0.8154	0.8249	0.8199	0.8309	0.8739	0.8704	0.8219	0.8784	0.8749
c500_d5000	0.7596	0.8489	0.8433	0.7750	0.8817	0.8819	0.7811	0.9008	0.9023	0.7823	0.9089	0.9129
c1000_d100	0.8241	0.8342	0.8342	0.8241	0.8342	0.8342	0.8241	0.8442	0.8442	0.8492	0.8593	0.8593
c1000_d1000	0.8754	0.8939	0.8934	0.8909	0.9275	0.9265	0.8849	0.9340	0.9335	0.8984	0.9455	0.9460
c1000_d5000	0.8338	0.9179	0.9195	0.8314	0.9285	0.9336	0.8397	0.9434	0.9460	0.8388	0.9472	0.9523
c1500_d100	0.9347	0.9347	0.9347	0.9548	0.9548	0.9548	0.9799	0.9749	0.9749	0.9799	0.9799	0.9799
c1500_d1000	—	—	—	—	—	—	—	—	—	—	—	0.9795
c1500_d5000	—	—	—	—	—	—	—	—	—	—	—	0.9889
c1800_d100	—	—	—	—	—	—	—	—	—	—	—	1.0000
c1800_d1000	—	—	—	—	—	—	—	—	—	—	—	0.9910
c1800_d5000	—	—	—	—	—	—	—	—	—	—	—	0.9966

The last 5 datasets were cut off in your output. You'd need to rerun those to get the full grid, but I've filled in the best results that were printed.

User: Hm? No they're not I have all the data.

GitHub Copilot: You're right! I apologize — can you paste the output for the last 4 datasets?

- c1500_d5000
- c1800_d100
- c1800_d1000
- c1800_d5000

Once I have those, I'll build the complete bagging table.

User: Yeah take a look at line 255 to 540.

GitHub Copilot: I can see the data goes up through c1500_d100 with the complete grid, but then c1500_d1000 cuts off mid-output. The summarized attachment file is truncated.

Can you paste the complete output for these datasets?

- c1500_d1000 (full)
- c1500_d5000
- c1800_d100
- c1800_d1000
- c1800_d5000

Once I have those, I'll build the complete bagging validation accuracy table for all 15 datasets.

User: Here

Processing dataset with 1500 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9795, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9825, Bagging Test F1 Score: 0.9824

Validation Accuracy for Bagging Classifier (n_estimators, max_depth):
n_estimators: 10, max_depth: None -> Validation Accuracy: 0.9720
n_estimators: 10, max_depth: 5 -> Validation Accuracy: 0.9520
n_estimators: 10, max_depth: 10 -> Validation Accuracy: 0.9720

n_estimators: 20, max_depth: None -> Validation Accuracy: 0.9760
n_estimators: 20, max_depth: 5 -> Validation Accuracy: 0.9575
n_estimators: 20, max_depth: 10 -> Validation Accuracy: 0.9760
n_estimators: 50, max_depth: None -> Validation Accuracy: 0.9770
n_estimators: 50, max_depth: 5 -> Validation Accuracy: 0.9565
n_estimators: 50, max_depth: 10 -> Validation Accuracy: 0.9770
n_estimators: 100, max_depth: None -> Validation Accuracy: 0.9795
n_estimators: 100, max_depth: 5 -> Validation Accuracy: 0.9605
n_estimators: 100, max_depth: 10 -> Validation Accuracy: 0.9795

Processing dataset with 1500 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9889, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9904, Bagging Test F1 Score: 0.9904

Validation Accuracy for Bagging Classifier (n_estimators, max_depth):
n_estimators: 10, max_depth: None -> Validation Accuracy: 0.9797
n_estimators: 10, max_depth: 5 -> Validation Accuracy: 0.9364
n_estimators: 10, max_depth: 10 -> Validation Accuracy: 0.9789
n_estimators: 20, max_depth: None -> Validation Accuracy: 0.9853
n_estimators: 20, max_depth: 5 -> Validation Accuracy: 0.9431
n_estimators: 20, max_depth: 10 -> Validation Accuracy: 0.9839
n_estimators: 50, max_depth: None -> Validation Accuracy: 0.9881
n_estimators: 50, max_depth: 5 -> Validation Accuracy: 0.9450
n_estimators: 50, max_depth: 10 -> Validation Accuracy: 0.9867
n_estimators: 100, max_depth: None -> Validation Accuracy: 0.9889
n_estimators: 100, max_depth: 5 -> Validation Accuracy: 0.9455
n_estimators: 100, max_depth: 10 -> Validation Accuracy: 0.9880

Processing dataset with 1800 clauses and 100 data points...
Best hyperparameters for bagging: {'n_estimators': 10, 'max_depth': None}, Validation Accuracy: 1.0000, Total Models Evaluated:

12
Bagging Test Accuracy: 0.9698, Bagging Test F1 Score: 0.9700

Validation Accuracy for Bagging Classifier (n_estimators, max_depth):
n_estimators: 10, max_depth: None -> Validation Accuracy: 1.0000
n_estimators: 10, max_depth: 5 -> Validation Accuracy: 1.0000
n_estimators: 10, max_depth: 10 -> Validation Accuracy: 1.0000
n_estimators: 20, max_depth: None -> Validation Accuracy: 1.0000
n_estimators: 20, max_depth: 5 -> Validation Accuracy: 1.0000
n_estimators: 20, max_depth: 10 -> Validation Accuracy: 1.0000
n_estimators: 50, max_depth: None -> Validation Accuracy: 1.0000
n_estimators: 50, max_depth: 5 -> Validation Accuracy: 1.0000
n_estimators: 50, max_depth: 10 -> Validation Accuracy: 1.0000
n_estimators: 100, max_depth: None -> Validation Accuracy: 0.9950
n_estimators: 100, max_depth: 5 -> Validation Accuracy: 0.9950
n_estimators: 100, max_depth: 10 -> Validation Accuracy: 0.9950

Processing dataset with 1800 clauses and 1000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9910, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9925, Bagging Test F1 Score: 0.9925

Validation Accuracy for Bagging Classifier (n_estimators, max_depth):
n_estimators: 10, max_depth: None -> Validation Accuracy: 0.9820
n_estimators: 10, max_depth: 5 -> Validation Accuracy: 0.9745
n_estimators: 10, max_depth: 10 -> Validation Accuracy: 0.9820
n_estimators: 20, max_depth: None -> Validation Accuracy: 0.9900
n_estimators: 20, max_depth: 5 -> Validation Accuracy: 0.9805
n_estimators: 20, max_depth: 10 -> Validation Accuracy: 0.9900
n_estimators: 50, max_depth: None -> Validation Accuracy: 0.9905
n_estimators: 50, max_depth: 5 -> Validation Accuracy: 0.9810
n_estimators: 50, max_depth: 10 -> Validation Accuracy: 0.9905

n_estimators: 100, max_depth: None -> Validation Accuracy: 0.9910
n_estimators: 100, max_depth: 5 -> Validation Accuracy: 0.9820
n_estimators: 100, max_depth: 10 -> Validation Accuracy: 0.9910

Processing dataset with 1800 clauses and 5000 data points...
Best hyperparameters for bagging: {'n_estimators': 100, 'max_depth': None}, Validation Accuracy: 0.9966, Total Models Evaluated: 12
Bagging Test Accuracy: 0.9976, Bagging Test F1 Score: 0.9976

Validation Accuracy for Bagging Classifier (n_estimators, max_depth):
n_estimators: 10, max_depth: None -> Validation Accuracy: 0.9945
n_estimators: 10, max_depth: 5 -> Validation Accuracy: 0.9739
n_estimators: 10, max_depth: 10 -> Validation Accuracy: 0.9942
n_estimators: 20, max_depth: None -> Validation Accuracy: 0.9954
n_estimators: 20, max_depth: 5 -> Validation Accuracy: 0.9806
n_estimators: 20, max_depth: 10 -> Validation Accuracy: 0.9952
n_estimators: 50, max_depth: None -> Validation Accuracy: 0.9962
n_estimators: 50, max_depth: 5 -> Validation Accuracy: 0.9802
n_estimators: 50, max_depth: 10 -> Validation Accuracy: 0.9963
n_estimators: 100, max_depth: None -> Validation Accuracy: 0.9966
n_estimators: 100, max_depth: 5 -> Validation Accuracy: 0.9801
n_estimators: 100, max_depth: 10 -> Validation Accuracy: 0.9963

GitHub Copilot: Perfect! Now I have all the data. Here's the complete **Validation Accuracy of Bagged Decision Trees** table organized:

Dataset	**10**			**20**			**50**			**100**		
max_depth	5	10	None	5	10	None	5	10	None	5	10	None
c300_d100	0.6382	0.6231	0.6231	0.6834	0.6633	0.6633	0.7286	0.7035	0.7035	0.6884	0.6834	0.6834
c300_d1000	0.8074	0.7589	0.7589	0.8289	0.7849	0.7819	0.8519	0.8464	0.8449	0.8534	0.8674	0.8689

Dataset	10			20			50			100		
c300_d5000	0.7877	0.8408	0.8161	0.7823	0.8595	0.8564	0.7938	0.8849	0.8864	0.7920	0.8943	0.8985
c500_d100	0.6281	0.5980	0.5980	0.6734	0.6583	0.6583	0.7387	0.7186	0.7186	0.7990	0.7889	0.7889
c500_d1000	0.7989	0.7739	0.7739	0.8154	0.8249	0.8199	0.8309	0.8739	0.8704	0.8219	0.8784	0.8749
c500_d5000	0.7596	0.8489	0.8433	0.7750	0.8817	0.8819	0.7811	0.9008	0.9023	0.7823	0.9089	0.9129
c1000_d100	0.8241	0.8342	0.8342	0.8241	0.8342	0.8342	0.8241	0.8442	0.8442	0.8492	0.8593	0.8593
c1000_d1000	0.8754	0.8939	0.8934	0.8909	0.9275	0.9265	0.8849	0.9340	0.9335	0.8984	0.9455	0.9460
c1000_d5000	0.8338	0.9179	0.9195	0.8314	0.9285	0.9336	0.8397	0.9434	0.9460	0.8388	0.9472	0.9523
c1500_d100	0.9347	0.9347	0.9347	0.9548	0.9548	0.9548	0.9799	0.9749	0.9749	0.9799	0.9799	0.9799
c1500_d1000	0.9520	0.9720	0.9720	0.9575	0.9760	0.9760	0.9565	0.9770	0.9770	0.9605	0.9795	0.9795
c1500_d5000	0.9364	0.9789	0.9797	0.9431	0.9839	0.9853	0.9450	0.9867	0.9881	0.9455	0.9880	0.9889
c1800_d100	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9950	0.9950	0.9950
c1800_d1000	0.9745	0.9820	0.9820	0.9805	0.9900	0.9900	0.9810	0.9905	0.9905	0.9820	0.9910	0.9910
c1800_d5000	0.9739	0.9942	0.9945	0.9806	0.9952	0.9954	0.9802	0.9963	0.9962	0.9801	0.9963	0.9966

That's your complete Table 1. Copy this directly into Obsidian.